

EXPLAIN – *show the execution plan of a statement*

La sentencia EXPLAIN no está definida en el SQL estándar, pero muchos motores la implementan.

EXPLAIN ANALYZE (explica y ejecuta)

<https://www.postgresql.org/docs/current/sql-explain.html>

Si queremos hacer un EXPLAIN ANALYZE sobre un INSERT, UPDATE, DELETE, CREATE TABLE AS, o EXECUTE sentencias sin que esto afecte a nuestros datos en la base, hay que usar BEGIN y ROLLBACK.

```
BEGIN;  
EXPLAIN ANALYZE ...;  
ROLLBACK;
```

BEGIN initiates a transaction block, that is, all statements after a BEGIN command will be executed in a single transaction until an explicit COMMIT or ROLLBACK is given. By default (without BEGIN), PostgreSQL executes transactions in "autocommit" mode, that is, each statement is executed in its own transaction and a commit is implicitly performed at the end of the statement (if execution was successful, otherwise a rollback is done).

The number of pages and rows is looked up in pg_class:

```
SELECT relpages, reltuples FROM pg_class WHERE relname = 'table';
```

relpages	reltuples
358	10000

La tabla pg_class del catálogo, contiene entre otras cosas cuantas páginas y cuantas tuplas tiene una tabla.

```
psql -U radiocut radiocut
```

```
radiocut=> CREATE TABLE t AS SELECT * FROM generate_series(1,100000);
```

```
radiocut=> \timing
```

```
Timing is on.
```

```
radiocut=> SELECT COUNT(*) FROM t;
```

```
count
```

```
-----
```

```
100000
```

```
(1 row)
```

```
Time: 12.232 ms
```

```
radiocut=> EXPLAIN ANALYZE SELECT COUNT(*) FROM t;
```

QUERY PLAN

```
-----  
Aggregate (cost=1855.06..1855.07 rows=1 width=0) (actual time=10.035..10.035 rows=1 loops=1)
```

```
  -> Seq Scan on t (cost=0.00..1572.65 rows=112965 width=0) (actual time=0.010..5.581 rows=100000 loops=1)
```

```
Planning time: 0.053 ms
```

```
Execution time: 10.088 ms
```

```
(4 rows)
```

```
Time: 11.798 ms
```

```
radiocut=> \d audiocut;
```

Table "public.audiocut"		
Column	Type	Modifiers
-----+-----+-----		
id	numeric	not null
slug	text	
radio_id	text	
radio_name	text	
show_id	text	
show_name	text	
start	timestamp with time zone	
length	numeric	
username	text	
description	text	
title	text	
created	timestamp with time zone	
play	numeric	

Indexes:

"audiocut_pk" PRIMARY KEY, btree (id)

"idx_audiocut_id" UNIQUE, btree (id)

```
radiocut=> explain select min(id) from audiocut;
```

QUERY PLAN

```
-----
Result  (cost=0.47..0.48 rows=1 width=0)
  InitPlan 1 (returns $0)
    -> Limit  (cost=0.42..0.47 rows=1 width=6)
          -> Index Only Scan using audiocut_pk on audiocut  (cost=0.42..8775.32 rows=185994 width=6)
                Index Cond: (id IS NOT NULL)
(5 rows)
```

Time: 1.451 ms

```
radiocut=> explain select min(play) from audiocut;  
              QUERY PLAN
```

```
-----  
Aggregate  (cost=22392.92..22392.93 rows=1 width=5)  
  -> Seq Scan on audiocut  (cost=0.00..21927.94 rows=185994 width=5)  
(2 rows)
```

Time: 2.364 ms

EXPLAIN

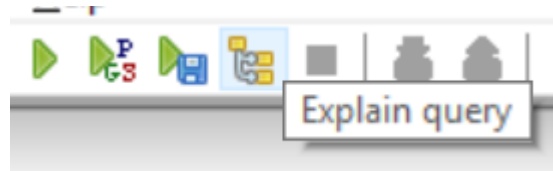
```
SELECT username, count(*), sum(play)  
FROM audiocut  
GROUP BY username  
ORDER BY sum(play) DESC  
LIMIT 10
```

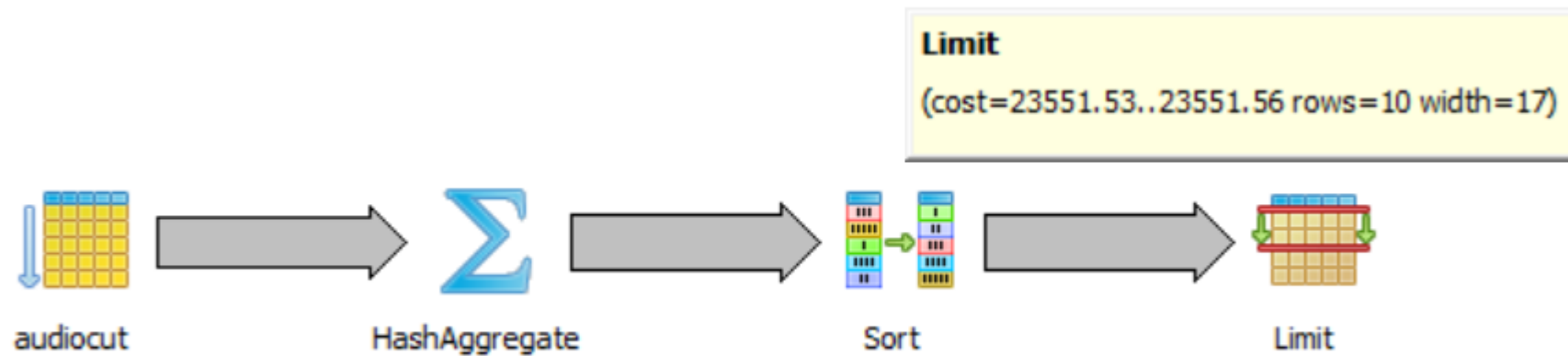
username	count(*)	sum(play)
RadioDelPlata	3487	906673
RadioAM750	3095	403449
TVShowUru1	94	317516
Eldestape	170	258478
detrasdeloquevemos	1295	196837

magustina	16	174438
monderosama	845	171611
LB24	2164	156872
josedelaradio	1070	142548
RADIONEF	3706	134944

QUERY PLAN

```
"Limit (cost=23551.53..23551.56 rows=10 width=17)"
"  -> Sort (cost=23551.53..23568.29 rows=6703 width=17)"
"      Sort Key: (sum(play)) DESC"
"      -> HashAggregate (cost=23322.90..23406.68 rows=6703 width=17)"
"          Group Key: username"
"          -> Seq Scan on audiocut (cost=0.00..21927.94 rows=185994 width=17)"
```





```
\timing
-- solo en el psql, no sirve en el pgAdmin

select count(*) from ar_amba_points
-- 532,733 registros
-- 31 msec
-- pero si lo volvemos a correr, puede dar otros valores, por ej. 61, 41, 30, ... ms

EXPLAIN select count(*) from ar_amba_points
"Aggregate (cost=11099.16..11099.17 rows=1 width=0)"
" -> Seq Scan on ar_amba_points (cost=0.00..9767.33 rows=532733 width=0)"

EXPLAIN ANALYZE select count(*) from ar_amba_points
```

```
"Aggregate (cost=11099.16..11099.17 rows=1 width=0) (actual time=103.080..103.080 rows=1 loops=1)"
"  -> Seq Scan on ar_amba_points (cost=0.00..9767.33 rows=532733 width=0) (actual time=0.014..58.494
rows=532733 loops=1)"
"Total runtime: 103.128 ms"
```

```
EXPLAIN VERBOSE select count(*) from ar_amba_points
```

```
"Aggregate (cost=11099.16..11099.17 rows=1 width=0)"
```

```
"  Output: count(*)"
```

```
"  -> Seq Scan on public.ar_amba_points (cost=0.00..9767.33 rows=532733 width=0)"
```

```
"    Output: id, geom"
```

```
EXPLAIN ANALYZE VERBOSE select count(*) from ar_amba_points
```

```
"Aggregate (cost=11099.16..11099.17 rows=1 width=0) (actual time=99.387..99.387 rows=1 loops=1)"
```

```
"  Output: count(*)"
```

```
"  -> Seq Scan on public.ar_amba_points (cost=0.00..9767.33 rows=532733 width=0) (actual time=0.012..56.671
rows=532733 loops=1)"
```

```
"    Output: id, geom"
```

```
"Total runtime: 99.427 ms"
```

```
ANALYZE [ VERBOSE ] [ table [ ( column [, ...] ) ] ]
```

```
ANALYZE VERBOSE ar_amba_points;
```

```
INFO: analyzing "public.ar_amba_points"
```

```
INFO: "ar_amba_points": scanned 4440 of 4440 pages, containing 532733 live rows and 0 dead rows; 30000 rows in
sample, 532733 estimated total rows
```

```
Query returned successfully with no result in 81 msec.
```

```
ANALYZE VERBOSE ar_amba_points(geom);
```

```
INFO: analyzing "public.ar_amba_points"
```

```
INFO: "ar_amba_points": scanned 4440 of 4440 pages, containing 532733 live rows and 0 dead rows; 30000 rows in sample, 532733 estimated total rows
```

```
Query returned successfully with no result in 71 msec.
```

```
ANALYZE VERBOSE ar_amba_points(id);
```

```
INFO: analyzing "public.ar_amba_points"
```

```
INFO: "ar_amba_points": scanned 4440 of 4440 pages, containing 532733 live rows and 0 dead rows; 30000 rows in sample, 532733 estimated total rows
```

```
Query returned successfully with no result in 41 msec.
```

How the Planner Uses Statistics

<https://www.postgresql.org/docs/current/static/planner-stats-details.html>

Query Planning

<https://www.postgresql.org/docs/current/static/runtime-config-query.html>

Los siguientes parámetros, por default están prendidos.

- `enable_bitmapscan`
- `enable_gathermerge`
- `enable_hashagg`

- enable_hashjoin
- enable_indexscan
- enable_indexonlyscan
- enable_material
- enable_mergejoin
- enable_nestloop
- enable_seqscan
- enable_sort
- enable_tidscan

Si queremos forzar a que no tome una estrategia, lo podemos apagar (temporalmente y sólo para nuestra sesión) de la siguiente manera:

```
set enable_seqscan = off;
```

También se pueden cambiar los costos (yo no los cambiaría)

Planner Cost Constants →

<https://www.postgresql.org/docs/current/static/runtime-config-query.html#RUNTIME-CONFIG-QUERY-CONSTANTS>

Algunos costos que mencionamos durante la clase:

```
cpu_tuple_cost = 0.01
```

```
seq_page_cost = 1
```

Fórmula (simple) para calcular el costo

$(\text{páginas} * \text{seq_page_cost}) + (\text{tuplas_a_retornar} * \text{cpu_tuple_cost})$

Genetic Query Optimization (GEQO) in PostgreSQL

<https://www.postgresql.org/docs/current/static/geqo-pg-intro.html>

BUFFERS hit & read

BUFFERS

Include information on buffer usage. Specifically, include the number of shared blocks hit, read, dirtied, and written, the number of local blocks hit, read, dirtied, and written, and the number of temp blocks read and written. A hit means that a read was avoided because the block was found already in cache when needed. Shared blocks contain data from regular tables and indexes; local blocks contain data from temporary tables and indexes; while temp blocks contain short-term working data used in sorts, hashes, Materialize plan nodes, and similar cases. The number of blocks dirtied indicates the number of previously unmodified blocks that were changed by this query; while the number of blocks written indicates the number of previously-dirtied blocks evicted from cache by this backend during query processing. The number of blocks shown for an upper-level node includes those used by all its child nodes. In text format, only non-zero values are printed. This parameter may only be used when ANALYZE is also enabled. It defaults to FALSE.

Ej.

```
EXPLAIN (ANALYZE,BUFFERS) select min(play) from audiocut;
```

```
"Aggregate (cost=6131.31..6131.32 rows=1 width=5) (actual time=74.741..74.742 rows=1 loops=1)"
```

```
"  Buffers: shared hit=716"
```

```
"    -> Index Only Scan using idx_audiocut_id_play on audiocut (cost=0.42..5666.33 rows=185994 width=5) (actual time=0.058..39.626 rows=185994 loops=1)"
```

```
"        Heap Fetches: 0"
```

```
"          Buffers: shared hit=716"
```

```
"Planning time: 0.521 ms"
```

```
"Execution time: 74.815 ms"
```

SELECT COUNT(*) FROM ...

Voy a analizar ahora otro caso, la base de datos con los casos de covid que tenemos alojada en el ITBA tiene al 9/08/2022 casi 30 millones de registros, una consulta simple puede llegar a tardar mucho y es importante que analicemos porque está pasando eso.

<https://covid19api.it.itba.edu.ar/api/v0.4.0/swagger/#/counter/countCases>

<https://covid19api.it.itba.edu.ar:443/api/v0.4.0/count> → { "count": 29.971.992 }

La consulta tardó más de 1 minuto (66.483 ms) en contar todos los registros con la API, pero debería tardar menos de 1 segundo, ¿qué estamos haciendo mal?

```
covid-api=# \timing
Timing is on.
```

```
covid-api=# select count(*) from "CovidCases";
 count
```

```
-----
```

```
29.971.992 (29.9 millones de registros)
(1 row)
```

```
Time: 62728.394 ms (01:02.728)
```

```
-- vamos a mirarlo con el EXPLAIN
covid-api=# explain select count(*) from "CovidCases";
```

QUERY PLAN

```
-----
Finalize Aggregate  (cost=933897.57..933897.58 rows=1 width=8)
->  Gather  (cost=933897.35..933897.56 rows=2 width=8)
    Workers Planned: 2
    ->  Partial Aggregate  (cost=932897.35..932897.36 rows=1 width=8)
        ->  Parallel Seq Scan on "CovidCases"  (cost=0.00..901691.48 rows=12482348 width=0)
```

JIT:

```
  Functions: 4
  Options: Inlining true, Optimization true, Expressions true, Deforming true
(8 rows)
```

Time: 1.966 ms

```
-- ahora con el EXPLAIN ANALYZE
```

```
covid-api=# explain analyze select count(*) from "CovidCases";
```

QUERY PLAN

```
-----
Finalize Aggregate  (cost=933897.57..933897.58 rows=1 width=8) (actual time=75228.534..75236.991 rows=1
loops=1)
->  Gather  (cost=933897.35..933897.56 rows=2 width=8) (actual time=75228.525..75236.985 rows=3 loops=1)
    Workers Planned: 2
```

```
Workers Launched: 2
-> Partial Aggregate (cost=932897.35..932897.36 rows=1 width=8) (actual
time=75190.589..75190.589 rows=1 loops=3)
      -> Parallel Seq Scan on "CovidCases" (cost=0.00..901691.48 rows=12482348 width=0) (actual
time=9.189..74537.711 rows=9990664 loops=3)
Planning Time: 0.354 ms
JIT:
  Functions: 8
  Options: Inlining true, Optimization true, Expressions true, Deforming true
  Timing: Generation 0.564 ms, Inlining 161.943 ms, Optimization 72.129 ms, Emission 12.635 ms, Total
247.271 ms
Execution Time: 75908.713 ms
(12 rows)

Time: 75929.583 ms (01:15.930)
```

Pero la aplicación a través de la API está haciendo este Query (que lo pude ver, porque active el log de las consultas que duran más de 1 segundo)

```
SELECT count(*) AS "count" FROM "CovidCases" AS "CovidCases" WHERE "CovidCases"."cuidado_intensivo" LIKE
 '%' AND "CovidCases"."fallecido" LIKE '%' AND "CovidCases"."asistencia_respiratoria_mecanica" LIKE '%' AND
"CovidCases"."clasificacion_resumen" LIKE '%' AND ("CovidCases"."fecha_apertura" >= '2000-01-01' AND
"CovidCases"."fecha_apertura" <= '2100-01-01');
```

Así que vamos a ver que dice el **EXPLAIN** con esta sentencia SQL.

Los LIKE '%' no hacen falta y el filtro de fechas válidas tampoco, cual es el overhead que se agrega con estos LIKEs y este control con las fechas?

Los índices de la tabla son los siguientes:

1. **CountIndex** btree (fallecido, clasificacion_resumen, cuidado_intensivo, asistencia_respiratoria_mecanica, fecha_apertura)
2. **ProvinceCountIndex** btree (carga_provincia_nombre, fallecido, clasificacion_resumen, cuidado_intensivo, asistencia_respiratoria_mecanica, fecha_apertura)
3. "ProvinceStatsIndex" btree (carga_provincia_nombre, fallecido, clasificacion_resumen, id_evento_caso)
4. **ProvinceSummaryIndex** btree (carga_provincia_nombre, fallecido, clasificacion_resumen, cuidado_intensivo, asistencia_respiratoria_mecanica, fecha_apertura, id_evento_caso)
5. **RawSummaryIndex** btree (fecha_apertura, id_evento_caso)
6. **StatsIndex** btree (fallecido, clasificacion_resumen, carga_provincia_nombre, id_evento_caso)
7. **SummaryIndex** btree (fallecido, clasificacion_resumen, cuidado_intensivo, asistencia_respiratoria_mecanica, fecha_apertura, id_evento_caso)

EXPLAIN

```
SELECT count(*) AS "count" FROM "CovidCases" AS "CovidCases" WHERE "CovidCases"."cuidado_intensivo" LIKE
 '%' AND "CovidCases"."fallecido" LIKE '%' AND "CovidCases"."asistencia_respiratoria_mecanica" LIKE '%' AND
"CovidCases"."clasificacion_resumen" LIKE '%' AND ("CovidCases"."fecha_apertura" >= '2000-01-01' AND
"CovidCases"."fecha_apertura" <= '2100-01-01');
```

QUERY_PLAN

=====

```
Finalize Aggregate  (cost=654531.00..654531.01 rows=1 width=8)
->  Gather  (cost=654530.79..654531.00 rows=2 width=8)
    Workers Planned: 2
    ->  Partial Aggregate  (cost=653530.79..653530.80 rows=1 width=8)
        ->  Parallel Index Only Scan using "CountIndex" on "CovidCases"  (cost=0.56..623631.96
rows=11959532 width=0)
```

```
Index Cond: ((fecha_apertura >= '2000-01-01'::text) AND (fecha_apertura <=
'2100-01-01'::text))
Filter: (((cuidado_intensivo)::text ~~ '%'::text) AND ((fallecido)::text ~~ '%'::text)
AND ((asistencia_respiratoria_mecanica)::text ~~ '%'::text) AND ((clasificacion_resumen)::text ~~
%'::text))
JIT:
  Functions: 6
  Options: Inlining true, Optimization true, Expressions true, Deforming true
(10 rows)
```

El costo pasó de 933897 a 654531, es decir disminuyó un 30%

Si solo miramos el control adicional de las fechas.

```
EXPLAIN SELECT count(*) AS "count" FROM "CovidCases" AS "CovidCases" WHERE "CovidCases"."fecha_apertura" >= '2000-01-01'
AND "CovidCases"."fecha_apertura" <= '2100-01-01';
```

```
Finalize Aggregate (cost=534935.68..534935.69 rows=1 width=8)
-> Gather (cost=534935.47..534935.68 rows=2 width=8)
    Workers Planned: 2
    -> Partial Aggregate (cost=533935.47..533935.48 rows=1 width=8)
        -> Parallel Index Only Scan using "CountIndex" on "CovidCases" (cost=0.56..504036.64 rows=11959532
width=0)
            Index Cond: ((fecha_apertura >= '2000-01-01'::text) AND (fecha_apertura <= '2100-01-01'::text))
JIT:
  Functions: 5
  Options: Inlining true, Optimization true, Expressions true, Deforming true
(9 rows)
```

El costo pasó de 933897 a 534935, es decir disminuyó un 43%

Si solo ponemos en el WHERE el LIKE %

```
EXPLAIN SELECT count(*) AS "count" FROM "CovidCases" AS "CovidCases" WHERE "CovidCases"."cuidado_intensivo" LIKE '%' AND  
"CovidCases"."fallecido" LIKE '%' AND "CovidCases"."asistencia_respiratoria_mecanica" LIKE '%' AND  
"CovidCases"."clasificacion_resumen" LIKE '%';
```

```
Finalize Aggregate (cost=511016.62..511016.63 rows=1 width=8)  
-> Gather (cost=511016.41..511016.62 rows=2 width=8)  
    Workers Planned: 2  
    -> Partial Aggregate (cost=510016.41..510016.42 rows=1 width=8)  
        -> Parallel Index Only Scan using "CountIndex" on "CovidCases" (cost=0.56..480117.58 rows=11959532  
width=0)  
            Filter: (((cuidado_intensivo)::text ~~ '%'::text) AND ((fallecido)::text ~~ '%'::text) AND  
((asistencia_respiratoria_mecanica)::text ~~ '%'::text) AND ((clasificacion_resumen)::text ~~ '%'::text))  
JIT:  
    Functions: 5  
    Options: Inlining true, Optimization true, Expressions true, Deforming true  
(9 rows)
```

El costo pasó de 933897 a 511017, es decir se incrementó un 45.3%

Por lo tanto, hay que tener cuidado con el LIKE % o con otros controles adicionales (el de las fechas), que nunca fue solicitado.

Por otro lado, si hacemos una consulta más específica, por ej. cuántos casos tengo que fallecieron por COVID19, al 25/03/2022 son 127.759 de 28.688.491, esa consulta se realiza en 26 milisegundos.

Si miramos el costo, es de 1202 frente a 391421 que cuesta contar TODOS los registros de la tabla "CovidCases".

```
covid-api=# explain select count(*) from "CovidCases" where fallecido = 'SI' and clasificacion_resumen = 'Confirmado';  
QUERY PLAN
```

```
-----  
Aggregate  (cost=1202.31..1202.32 rows=1 width=8)  
->  Index Only Scan using "CountIndex" on "CovidCases"  (cost=0.56..1086.56 rows=46300 width=0)  
    Index Cond: ((fallecido = 'SI'::text) AND (clasificacion_resumen = 'Confirmado'::text))  
(3 rows)
```

```
covid-api=# explain analyze select count(*) from "CovidCases" where fallecido = 'SI' and clasificacion_resumen =  
'Confirmado';
```

```
QUERY PLAN  
-----  
Aggregate  (cost=1202.31..1202.32 rows=1 width=8) (actual time=26.789..26.789 rows=1 loops=1)  
->  Index Only Scan using "CountIndex" on "CovidCases"  (cost=0.56..1086.56 rows=46300 width=0) (actual  
time=0.051..17.203 rows=127759 loops=1)  
    Index Cond: ((fallecido = 'SI'::text) AND (clasificacion_resumen = 'Confirmado'::text))  
    Heap Fetches: 0  
Planning Time: 0.085 ms  
Execution Time: 26.806 ms  
(6 rows)
```

```
select count(*) from graduados  
-- 8532
```

titulo	apellido	nombre
Ingeniero Industrial	Moreyra Maida	Solange Micaela
Ingeniero en Petróleo	Gilardone	Agustina
Ingeniero Electrónico	Burna Macol	Daniel
Ingeniero Industrial	Schojet	Maya
Ingeniero Industrial	Lugones	Facundo Lorenzo
..	..	..

```
select count(*) from graduados
-- 8532
```

```
select * from graduados limit 10
```

```
explain analyse
  select *
  from graduados
  where apellido = 'Lugones' and nombre = 'Facundo Lorenzo'
```

```
-- sin indice
```

```
Seq Scan on graduados  (cost=0.00..208.98 rows=1 width=43) (actual time=0.022..2.099 rows=1 loops=1)
```

```
Planning Time: 0.104 ms
```

```
Execution Time: 2.115 ms
```

```
CREATE INDEX graduados_a_idx ON graduados (apellido);
```

```
explain analyse
  select *
  from graduados
  where apellido = 'Lugones' and nombre = 'Facundo Lorenzo'
```

Index Scan using graduados_a_idx on graduados (cost=0.29..8.30 rows=1 width=43) (actual time=0.131..0.158 rows=1 loops=1)

```
  Index Cond: (apellido = 'Lugones'::text)
  Filter: (nombre = 'Facundo Lorenzo'::text)
  Rows Removed by Filter: 1
```

Planning Time: 1.270 ms

Execution Time: 0.177 ms

```
CREATE INDEX graduados_an_idx ON graduados (apellido, nombre);
```

```
explain analyse
  select *
  from graduados
  where apellido = 'Lugones' and nombre = 'Facundo Lorenzo'
```

Index Scan using graduados_an_idx on graduados (cost=0.29..8.30 rows=1 width=43) (actual time=0.083..0.084 rows=1 loops=1)

```
  Index Cond: ((apellido = 'Lugones'::text) AND (nombre = 'Facundo Lorenzo'::text))
```

Planning Time: 0.605 ms

Execution Time: 0.108 ms

```
explain analyse
  select *
  from graduados
  where
  nombre = 'Facundo Lorenzo'
```

```
-- no usa ningun indice, hace un escaneo secuencial
Seq Scan on graduados  (cost=0.00..187.65 rows=1 width=43) (actual time=0.022..2.033 rows=1 loops=1)
  Filter: (nombre = 'Facundo Lorenzo'::text)
  Rows Removed by Filter: 8531
Planning Time: 0.147 ms
Execution Time: 2.048 ms

explain analyse
  select *
  from graduados
  where apellido like '%' and nombre = 'Facundo Lorenzo'

Seq Scan on graduados  (cost=0.00..208.98 rows=1 width=43) (actual time=0.040..2.418 rows=1 loops=1)
  Filter: ((apellido ~~ '%'::text) AND (nombre = 'Facundo Lorenzo'::text))
  Rows Removed by Filter: 8531
Planning Time: 0.202 ms
Execution Time: 2.527 ms

CREATE INDEX graduados_n_idx ON graduados (nombre);

drop index graduados_an_idx

explain analyse
  select *
  from graduados
  where apellido = 'Lugones' and nombre = 'Facundo Lorenzo'

Index Scan using graduados_n_idx on graduados  (cost=0.29..8.30 rows=1 width=43) (actual time=0.193..0.197
rows=1 loops=1)
  Index Cond: (nombre = 'Facundo Lorenzo'::text)
  Filter: (apellido = 'Lugones'::text)
Planning Time: 2.369 ms
```

Execution Time: 0.240 ms

```
explain analyse
  select *
  from graduados
  where nombre = 'Facundo Lorenzo' and apellido = 'Lugones'
```

Index Scan using graduados_n_idx on graduados (cost=0.29..8.30 rows=1 width=43) (actual time=0.060..0.063 rows=1 loops=1)

Index Cond: (nombre = 'Facundo Lorenzo'::text)

Filter: (apellido = 'Lugones'::text)

Planning Time: 0.326 ms

Execution Time: 0.105 ms

```
select count(*) from (select distinct apellido from graduados) as foo
-- 6.332
```

```
select count(*) from (select distinct nombre from graduados) as foo
-- 4.318
```

```
select nombre, apellido, length(nombre) from graduados order by 3 limit 10
```

```
explain analyse
  select *
  from graduados
  where nombre = 'Luz' and apellido = 'Wetzler Malbrán'
```

Performance Tips > Using EXPLAIN

<https://www.postgresql.org/docs/current/using-explain.html>